

Code-Layout, Readability and Reusability

Date	30 September 2026
Team ID	PER206AIFITTRACK
Project Name	AI-FitTrack-API
Maximum Marks	2 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist & Directory Architecture:

Directory / Component	Design Role in System	Reusability & Code Readability Conventions
server.js	Application Root	Initializes Express application, connects to MongoDB, mounts root middleware (cors, express.json), and binds REST route prefixes.
config/db.js	Database Connection Module	Encapsulates Mongoose connection logic with async/await, URI parameterization, and automated reconnect error listeners.
models/User.js	Mongoose User Schema	Defines user credentials, profile fields, fitness goals, and schema-level validation constraints.
models/Workout.js	Mongoose Workout Schema	Defines workout logs with enum categories (Cardio, Strength, HIIT), MET duration, and ObjectId reference to User.
middleware/authMiddleware.js	Reusable JWT Route Guard	Intercepts requests, extracts Authorization Bearer header, validates token signature, and binds user payload to req.user.
controllers/authController.js	Authentication Controller	Handles register, login, and profile retrieval logic with bcryptjs password hashing and JWT token generation.
controllers/workoutController.js	Workout CRUD Controller	Implements create, read, update, delete, and multi-parameter query filtering with lean database serialization.
controllers/aiController.js	Google Gemini AI Controller	Orchestrates prompt construction with user profile context, calls gemini-1.5-flash, and parses structured output.
routes/*.js	Express Router Modules	Modular route definitions cleanly decoupling URL path mapping from controller execution logic.

Readability & Code Style Best Practices Enforced:

Coding Standard	Enforcement in AI-FitTrack-API Architecture
Asynchronous Programming	Strict use of async/await and try/catch blocks; zero unhandled promise rejections.
Environment Isolation	Sensitive API keys, MongoDB connection strings, and JWT secrets isolated in .env.
Naming Conventions	camelCase for variables/functions, PascalCase for Mongoose models, UPPERCASE for constants.
Centralized Error Format	Standardized JSON error envelope ({ success: false, message: ... }) returned across all routes.